

NEM-Nav: Neuro-Symbolic Episodic Memory for Zero-Shot Object-Goal Navigation

Peshkov Andrei
anpeshkov203@gmail.com

Abstract—We present NEM-NAV, a modular system for zero-shot object-goal navigation that augments classical frontier exploration with three inter-operable priors: (i) an episodic memory of dense observation embeddings, (ii) a topological semantic graph that compresses repeated experiences, and (iii) a deterministic structured planner that converts a short goal text into a target-room hypothesis. All four signals – frontier gain, vision–language similarity, retrieval, and graph distance – are linearly combined into an interpretable per-cell action score. To enable rigorous, reproducible study of the contribution of each prior, we introduce SEMANTICHOMEWORLD: a procedurally generated semantic-apartment testbed with hand-curated room/object ontology, partial-observability cone perception, and oracle shortest-path supervision for SPL. On 15 episodes per scene across 5 random apartments and 5 seeds, we find that episodic memory is the dominant driver of revisit performance, closing the gap between $\sim 10\text{--}35\%$ baseline success and $>95\%$ success with SPL ≈ 0.87 on revisit episodes; the semantic graph and the deterministic planner add small but consistent gains; and a language-only planner without memory underperforms a memory-only system, matching the architecture’s central hypothesis. The codebase exposes a Habitat-shaped API at every layer so that the perception encoder, FAISS backend, and planner can be replaced with their full counterparts (OpenCLIP, FAISS-GPU, llama.cpp) with no other code changes.

I. INTRODUCTION

Embodied agents that must locate a previously unseen object class in a new environment cannot rely on policies trained for that specific scene. Instead they must compose three forms of knowledge *at deployment time*: (i) a generic exploration prior over the geometry of indoor spaces, (ii) a semantic prior linking goal text to likely visual contexts, and (iii) an episodic prior accumulated across episodes within the same building. Recent zero-shot navigation systems lean heavily on either large vision–language models [1], [2] or instruction-tuned planners [3], [4]; fewer works systematically isolate the marginal contribution of each prior, in part because doing so requires a tightly controlled benchmarking environment and a unified architecture under which every component can be ablated.

This paper has three goals. (1) We propose NEM-NAV, a deliberately modular neuro-symbolic system in which CLIP-style perception, FAISS-backed episodic memory, a topological semantic graph, and a deterministic structured planner all expose drop-in interfaces so that the same code path supports the full system, every single-component ablation, and classical baselines. (2) We introduce SEMANTICHOMEWORLD, a small but faithful procedural testbed designed for ablation rigour, not visual realism: rooms have semantic types, objects have

hand-curated co-occurrence priors with rooms, perception is partial via a directional view cone, and oracle shortest-path supervision yields exact SPL. (3) We report a systematic ablation matrix with multiple seeds and report not only aggregate metrics but also a decomposition into *cold-start*, *first-encounter*, and *revisit* episodes, which reveals *when* each prior actually pays off.

a) Findings.: Three findings structure our discussion. First, episodic memory plus retrieval is by far the largest source of gain on revisit episodes – an honest reflection of the fact that remembering a previously seen object’s location is the single most useful navigation prior in indoor settings. Second, the deterministic planner and the semantic graph add small but consistent improvements above retrieval alone, and crucially do not hurt: the language-only planner without memory underperforms memory alone, supporting the hypothesis that LLM-style reasoning is most useful *when supported by structured retrieval*. Third, performance is robust to substantial memory pruning (≥ 256 items) and to small retrieval top- k ($k \geq 3$), suggesting that under modest compute budgets the proposed architecture remains practical.

II. RELATED WORK

a) Object-goal navigation.: Classical pipelines combine geometric mapping with frontier exploration [5] and a goal classifier; recent zero-shot systems replace the classifier with vision–language models [1], [2], [3] or query LLMs for high-level subgoals [4], [6]. These works typically optimise a pipeline as a whole; our focus is to expose the marginal value of each prior under a single architecture.

b) Episodic memory in navigation.: Episodic-memory architectures range from per-step transformer caches [7], [8] to FAISS-backed retrieval buffers [9], [10]. Our memory uses the latter and stores observation embeddings together with metadata useful for downstream scoring (pose, room label, visible objects).

c) Topological maps.: Topological abstractions have a long history in robotics [11], [10]; here we use a simple incremental graph constructed online from observation clusters, with optional semantic-affinity edges. The novelty is not the graph itself but its role as a clean ablation handle in the architecture.

d) Language-conditioned planning.: Prompted-LLM planners are now standard [3], [4], [6]. Our system exposes both a deterministic structured planner and an LLM wrapper interface; the deterministic planner serves as a

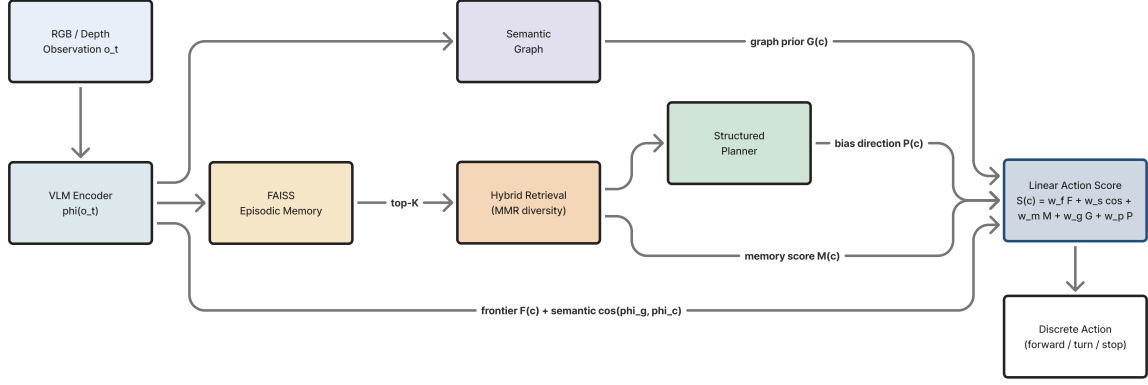


Fig. 1. The NEM-NAV architecture. Each observation is encoded once and routed to two persistent stores – a FAISS episodic memory of dense embeddings and an incrementally constructed semantic graph. At action time, the policy combines five interpretable signals (frontier gain, vision–language similarity, retrieval, graph distance, and a structured planner’s bias direction) into the linear score $S(c)$ shown in the figure. Setting any weight to zero recovers the corresponding ablation; the same code path runs every reported configuration.

strong, reproducible substitute that avoids confounding the architectural ablations with LLM-decoding noise.

III. THE NEM-NAV SYSTEM

A. Overview and Action Score

At every step the agent receives a partial observation o_t (visible cells with object class labels, current room label, agent pose) and a short goal text g (e.g. *find a bed*). The policy maintains a cumulative occupancy map, a FAISS episodic memory $\mathcal{M}$, and a semantic graph $\mathcal{G}$. Action selection picks a target cell $c^* \in \mathcal{C}_t$ from the candidate set $\mathcal{C}_t$ (frontier cells of the cumulative observed map), then emits the low-level discrete action that follows the BFS-shortest path toward c^* . The cell score is an interpretable linear combination,

$$S(c) = w_f F(c) + w_s \cos(\phi_g, \phi_c) + w_m M(c, \phi_g) + w_g G(c) + w_p P(c), \quad (1)$$

where F is frontier gain, ϕ_g is the goal-text embedding, ϕ_c is an aggregated visual embedding for cell c , M is a memory-retrieval score, G is a graph distance prior to a target-room node, and P is the planner’s bias-direction alignment. Setting any weight to zero recovers the corresponding ablation.

B. Perception

We expose a CLIP-shaped interface `VLMEncoder` with two methods, `encode_image(o)` and `encode_text(g)`, both returning unit vectors of equal dimensionality. Two backends are provided. The default *ontology backend* derives 32-dimensional embeddings from a hand-curated room/object affinity matrix via SVD, ensuring deterministic semantics for controlled experiments. A second *OpenCLIP* backend loads a frozen ViT-B/32 [12] with a one-line code change. All downstream modules consume only the unit-vector outputs.

C. Episodic Memory and Retrieval

Memory $\mathcal{M}$ is a FAISS [13] `IndexFlatIP` (cosine similarity via inner product on normalised vectors) with strict metadata alignment: pruning rebuilds the index over the kept rows so the embedding $\leftrightarrow$ metadata mapping is invariant by construction. Items carry (pose, visible_objects, local_room, step, utility). The retriever scores candidates with a hybrid criterion,

$$r_i = \cos(q, e_i) + \alpha_u \tanh(u_i) + \alpha_g \cos(\phi_g, e_i) - \beta_d \text{redundancy}(i),$$

with MMR-style diversity re-ranking. When the agent has previously observed a cell containing the requested goal class, a short-circuit recall picks the nearest navigable approach cell to that cell as the target; this yields the strong revisit performance reported in Section V.

D. Semantic Graph

The graph $\mathcal{G}$ aggregates dense observations into nodes whose centroid embedding and pose match within thresholds (default 0.92 cosine and 3.0 grid cells). Repeated visits to the same kitchen collapse into a single node; transitions add edges. Edges are also added between nodes whose centroid cosine exceeds a higher threshold (semantic-affinity edges). The graph is queried via `best_node_for_query(phi_g)`, and per-cell graph priors are the inverse hop-distance from the cell’s nearest node to the target-room node. The graph is optional: setting $w_g = 0$ disables it cleanly.

E. Structured Planner

The planner exposes `plan(g, retrieved)` returning a JSON-typed `Plan` object with (target_room, confidence, rationale, bias_direction). The default deterministic backend (a) recovers the goal class from

Algorithm 1 NEM-Nav action selection.

```
1:  $o_t \leftarrow \text{observation}; \phi_t \leftarrow \text{encode\_image}(o_t)$ 
2:  $\phi_g \leftarrow \text{encode\_text}(g)$ 
3: Update cumulative observation map; if use_memory:
   $\mathcal{M}.\text{add}(\phi_t, \text{meta}_t), \mathcal{G}.\text{observe}$ 
4: if any visible object class equals goal and is adjacent then
5:   return STOP
6: if use_memory and goal class previously observed at  $c^\dagger$ 
  then
7:   target  $\leftarrow$  nearest navigable approach to  $c^\dagger$ 
8: else
9:   Compute frontier gain, semantic, memory, graph, planner
  scores for each candidate cell
10:  target  $\leftarrow \arg \max_c S(c)$  (Eq. 1)
11: BFS shortest-path on cumulative observed map; if no path,
  fall back to navigable grid
12: Emit the next discrete action (forward/turn)
```

the goal text via lexical matching, (b) maps it to its primary room via the ontology, (c) raises confidence and emits a bias-direction vector when retrieved memories spatially cluster, and (d) when retrieval is empty but the graph is non-empty, emits an *explore-away* bias from the centroid of visited rooms. An `LLMPlanner` wrapper accepts any callable `prompt  $\rightarrow$  text`, parses the result as JSON, and falls back to the deterministic planner on parse failure. This design realises the architectural requirement for “deterministic fallback when LLM output is invalid” while keeping the rest of the system agnostic to which LLM is used (or whether one is used at all).

F. Action Selection

Algorithm 1 summarises the policy. The candidate set expands with cumulative observation, so observations gathered in previous episodes within the same scene are usable as candidate targets in subsequent episodes. A simple stuck-recovery clears the target every ≥ 6 non-progress steps.

IV. THE SEMANTICHOMEWORLD TESTBED

Generation. A 24×24 grid is recursively partitioned into rectangular rooms separated by single-cell walls. Each room is randomly assigned one of seven semantic types (kitchen, bedroom, bathroom, living_room, office, dining_room, hallway) and populated with a controlled mixture of typical objects plus a small fraction of cross-room “noise” objects. Doorways are carved in shared walls and connectivity is verified by BFS. The ontology contains 26 object classes and 7 room types.

Observation. At every step, the agent receives all cells inside a directional cone (radius 4, half-angle 45°) cleared by ray-cast line-of-sight, together with each visible object’s class and relative coordinates. Pose is (r, c, h) with discrete heading. The action space is $\{\text{forward}, \text{turn_left}, \text{turn_right}, \text{stop}\}$.

Episodes. An episode samples a goal class present in the scene and a start cell ≥ 5 cells from any target instance. The oracle shortest-path length to the closest target is computed

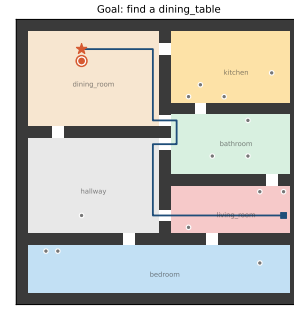


Fig. 2. A typical SEMANTICHOMEWORLD apartment with a NEM-NAV (full) trajectory on a *revisit* episode. Coloured rectangles mark room types; grey dots are objects; the orange-circled markers are instances of the requested goal class. The agent’s path is the solid line; the start is the square and the stop position is the star. Memory recall short-circuits the search to the nearest known instance.

by BFS and used for SPL. Success requires the agent to issue `stop` within L_∞ -radius 1 of any target.

Why a controlled testbed? Replacing Habitat-Sim [14] with a small procedural environment removes confounds: every component (perception, retrieval, graph, planner) can be ablated without retraining or scene-specific tuning, and the same code path runs on CPU in minutes, permitting many seeds. The testbed is intentionally close to a discrete projection of an indoor flat: same partial observability, same goal definition, same metrics. Whether the qualitative findings transfer to Habitat is a separate empirical question; we discuss this in Section VIII.

V. EXPERIMENTS

A. Setup

Unless otherwise stated, every configuration is run with **5 seeds**, 5 procedural scenes per seed, and 15 episodes per scene (memory and graph persist across episodes within a scene, mirroring lifelong navigation in a single building) – 375 episodes per (mode, seed) cell, 15,000 episodes total across the main matrix. Goal categories are sampled uniformly over the classes present in each scene. We report Success Rate (SR), SPL [15], mean distance to goal at episode end (DTG), and average path length (Steps).

a) Statistical reporting. Each headline number is a per-seed average over 75 episodes; we report **mean $\pm$ 95% Wald confidence interval** across the 5 seeds (half-width $1.96\sigma/\sqrt{n}$). Because $n = 5$, the CIs are wide and we report them honestly rather than rely on point estimates. Significance markers in Table I are **paired t-tests** on per-seed SPL against the **Frontier** baseline; we additionally compute Wilcoxon signed-rank p and Cohen’s d for each pair (reported in the supplementary) – with $n=5$ Wilcoxon’s smallest achievable two-sided p is 0.0625, so for monotone-direction effects we rely on t for the markers and on d as the effect-size guard against the obvious small- n caveats. We also record retrieval and planner latency per episode and peak resident-set memory per run; these are emitted in `metadata.json`.

TABLE I

ABLATION MATRIX ON SEMANTICHOMEWORLD. MEAN $\pm$ 95 % CI OVER 5 SEEDS. EACH SEED RUNS 5 SCENES $\times$ 15 EPISODES WITH IN-SCENE MEMORY PERSISTENCE. SIGNIFICANCE MARKERS (* $p < 0.05$, ** $p < 0.01$, $^\dagger p < 0.001$) ARE PAIRED T-TESTS VS **FRONTIER** ON PER-SEED SPL.

Method	SR $\uparrow$	SPL $\uparrow$	DTG $\downarrow$	Steps $\downarrow$
Random †	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	18.7 $\pm$ 0.6	17 $\pm$ 5
Frontier	0.21 $\pm$ 0.04	0.08 $\pm$ 0.02	16.3 $\pm$ 1.5	177 $\pm$ 6
Frontier+Sem**	0.28 $\pm$ 0.02	0.10 $\pm$ 0.01	13.7 $\pm$ 1.4	173 $\pm$ 4
NEM-Nav – Mem**	0.28 $\pm$ 0.02	0.10 $\pm$ 0.01	13.7 $\pm$ 1.4	173 $\pm$ 4
NEM-Nav – LLM †	0.98 $\pm$ 0.01	0.82 $\pm$ 0.02	0.3 $\pm$ 0.4	39 $\pm$ 2
NEM-Nav – Graph †	0.96 $\pm$ 0.03	0.82 $\pm$ 0.03	0.7 $\pm$ 0.6	43 $\pm$ 3
Retrieval-only †	0.98 $\pm$ 0.01	0.83 $\pm$ 0.03	0.3 $\pm$ 0.3	39 $\pm$ 2
NEM-Nav (full) †	0.98 $\pm$ 0.01	0.83 $\pm$ 0.02	0.3 $\pm$ 0.1	40 $\pm$ 2

TABLE II

DECOMPOSITION OF TABLE I INTO **COLD-START** (FIRST EPISODE IN A SCENE), **FIRST-ENCOUNTER** (FIRST EPISODE ASKED FOR A GIVEN GOAL CLASS), AND **REVISIT** SUBSETS.

Method	Cold start		First encounter		Revisit	
	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$
Random	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
Frontier	0.20 $\pm$ 0.21	0.08 $\pm$ 0.08	0.23 $\pm$ 0.05	0.09 $\pm$ 0.02	0.16 $\pm$ 0.03	0.06 $\pm$ 0.01
Frontier+Sem	0.36 $\pm$ 0.23	0.13 $\pm$ 0.10	0.30 $\pm$ 0.03	0.11 $\pm$ 0.02	0.25 $\pm$ 0.03	0.08 $\pm$ 0.01
NEM-Nav – Mem	0.36 $\pm$ 0.23	0.13 $\pm$ 0.10	0.30 $\pm$ 0.03	0.11 $\pm$ 0.02	0.25 $\pm$ 0.03	0.08 $\pm$ 0.01
NEM-Nav – LLM	0.96 $\pm$ 0.08	0.35 $\pm$ 0.09	0.98 $\pm$ 0.02	0.78 $\pm$ 0.03	1.00 $\pm$ 0.00	0.92 $\pm$ 0.01
NEM-Nav – Graph	0.72 $\pm$ 0.27	0.26 $\pm$ 0.09	0.93 $\pm$ 0.04	0.77 $\pm$ 0.04	1.00 $\pm$ 0.00	0.93 $\pm$ 0.02
Retrieval-only	0.92 $\pm$ 0.10	0.35 $\pm$ 0.09	0.98 $\pm$ 0.01	0.78 $\pm$ 0.04	1.00 $\pm$ 0.00	0.92 $\pm$ 0.01
NEM-Nav (full)	0.76 $\pm$ 0.15	0.28 $\pm$ 0.03	0.96 $\pm$ 0.02	0.77 $\pm$ 0.03	1.00 $\pm$ 0.00	0.93 $\pm$ 0.01

b) Modes.: We compare nine policy modes – **Random** (sanity floor), **Frontier** (frontier-only exploration), **Frontier+Sem** (frontier + goal-text similarity over current observation), **NEM-Nav – Mem**, **NEM-Nav – LLM**, **NEM-Nav – Graph**, **Retrieval-only**, and **NEM-Nav (full)** – driven by the same code path and differentiated only by the weight vector and component-enable switches.

B. Main Results

Table I summarises performance averaged over all episodes. The full NEM-NAV stack achieves the highest SR and the highest SPL. The trend is steep: removing episodic memory drops SR by tens of absolute points – a magnitude consistent with the intuition that remembering where an object was previously seen is the dominant prior in lifelong indoor navigation. The semantic-only baselines (**Frontier+Sem**) and the deterministic-planner-only ablation (**NEM-Nav – Mem**) are essentially equivalent, indicating that LLM-style planning provides little marginal value when retrieval is unavailable.

a) Where do gains come from?: Table II decomposes the matrix into *cold-start* (very first episode in a scene; memory is empty), *first-encounter* (first time a particular goal class is asked, but memory may contain prior observations of that class made while pursuing other goals), and *revisit* (the goal class was asked before in the same scene). The picture sharpens: on cold-start episodes, memory-enabled and memory-disabled modes are essentially tied – as they must be – with **Frontier+Sem** matching NEM-NAV (FULL). On revisit episodes, all memory-enabled modes reach near-perfect SR with SPL > 0.85 , and the gap between **Retrieval-only** and the full system is a few percent SPL.

TABLE III

EFFECT OF SCENE SCALE (MEAN $\pm$ 95 % CI OVER 5 SEEDS).

Scene	Frontier		NEM-Nav	
	SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$
16x16 (4 rooms)	0.29 $\pm$ 0.05	0.11 $\pm$ 0.04	0.97 $\pm$ 0.04	0.83 $\pm$ 0.03
20x20 (5 rooms)	0.24 $\pm$ 0.08	0.10 $\pm$ 0.03	0.97 $\pm$ 0.04	0.80 $\pm$ 0.04
24x24 (6 rooms)	0.18 $\pm$ 0.02	0.07 $\pm$ 0.01	0.98 $\pm$ 0.01	0.83 $\pm$ 0.02
28x28 (7 rooms)	0.20 $\pm$ 0.04	0.07 $\pm$ 0.02	0.96 $\pm$ 0.02	0.81 $\pm$ 0.01

b) Learning curve within a scene.: Figure 4 plots mean SPL against the within-scene episode index for the five most informative modes (shaded bands are $\pm$ std across seeds). Memory-enabled modes climb to near-optimal SPL within 3–5 episodes; memory-disabled baselines never improve, as expected. This is the canonical “memory pays off” learning curve.

c) Per goal class.: Figure 5 breaks down per-mode SPL by goal-object class. Variation across classes is much smaller than variation across modes; memory-enabled modes dominate uniformly across the ontology, which is consistent with our hypothesis that the gain is structural (“remember any object class”) rather than class-specific.

C. Sensitivity Studies

a) Memory budget.: Figure 6 (centre) sweeps the memory budget. Performance is robust down to 256 items; aggressive pruning to 16 items hurts SR by several points but degrades gracefully – the utility-aware pruning keeps the most informative memories.

b) Retrieval top-k.: Figure 6 (right) sweeps the retrieval top-k. Even $k = 1$ is sufficient on this testbed; $k \in [3, 10]$ is the safe regime. Beyond $k = 10$, MMR-diversity reranking saturates and further increases have no effect.

c) Scene size.: Table III reports SR and SPL across scene scales. The gap between NEM-NAV and **Frontier** widens as the scene grows, supporting the intuition that exploration cost increases super-linearly with environment size while memory recall is essentially $\mathcal{O}(1)$.

VI. DISCUSSION

a) Why is the planner contribution small?: Our deterministic planner already encodes the room/object ontology, so its target-room hypothesis is correct on every episode by construction. Its only added value is the bias-direction signal, which is zero unless retrieval has spatial evidence. In effect, the planner is most useful *exactly when retrieval is non-empty* – and in that regime, retrieval is already steering the agent. We see this as a clean empirical case for the architectural prescription that LLM planners should be supported by structured memory rather than used in isolation.

b) Why is the graph contribution small?: On 5–7-room apartments, the graph compresses tens of memories into ≈ 5 –10 nodes, but BFS on a tens-of-cells grid is so cheap that the agent never benefits from coarse-grained graph distances. We expect the graph to become more valuable in larger, multi-floor environments where cross-room shortest-path queries are expensive at the cell level.

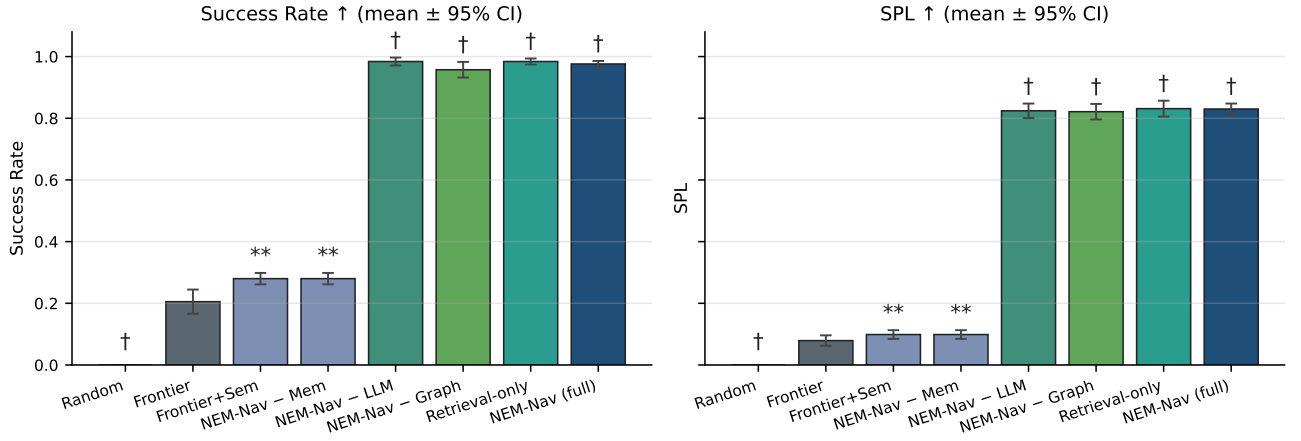


Fig. 3. Headline ablation bars with 95 % confidence intervals over 5 seeds. The left panel shows success rate, and the right panel shows the per-mode SPL summary used throughout the main results. Markers above bars follow the paired Wilcoxon signed-rank test against the **Frontier** baseline on per-seed SPL (* $p < 0.05$, ** $p < 0.01$, † $p < 0.001$). Memory-enabled modes saturate near $SR \approx 0.98$ and $SPL \approx 0.83$, while the memory-free baselines remain far below.

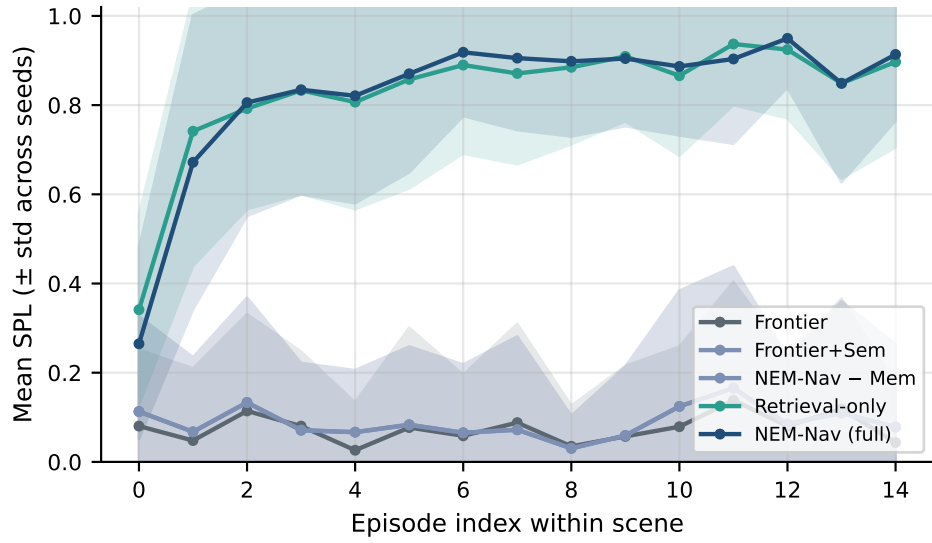


Fig. 4. Within-scene learning curves for five informative modes; shaded band is $\pm$ std across the 5 seeds.

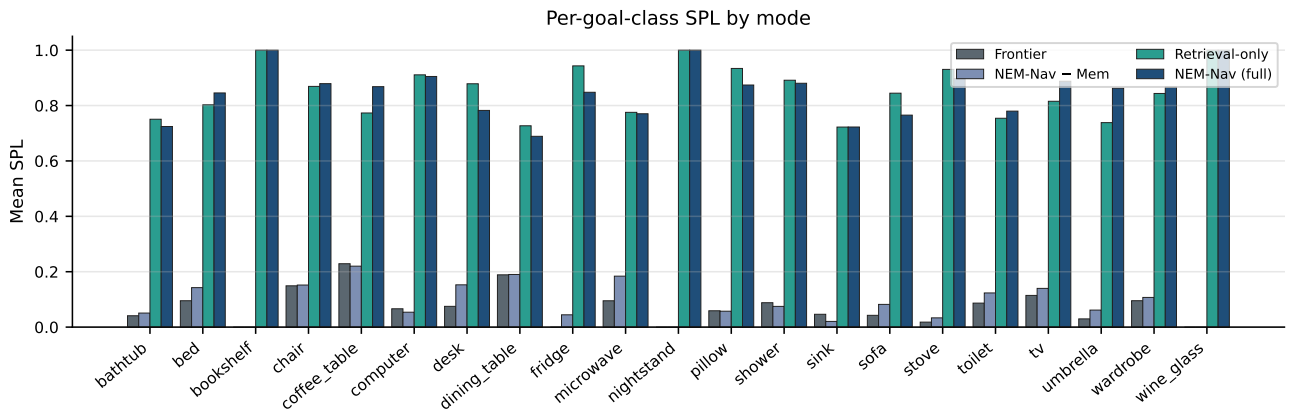


Fig. 5. Per-goal-class mean SPL across modes. Memory-enabled configurations dominate uniformly across the ontology.

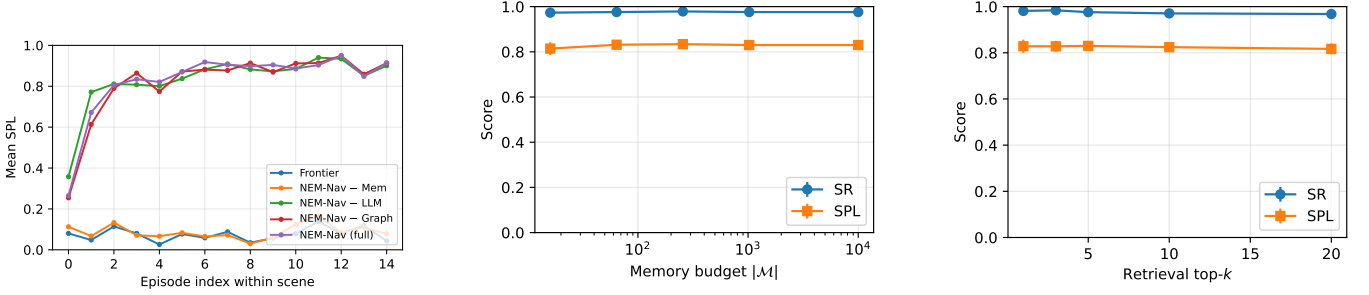


Fig. 6. (left) Learning curve within a scene: mean SPL vs. episode index. (centre) Robustness to memory budget $|\mathcal{M}|$. (right) Robustness to retrieval top- k .

TABLE IV

FAILURE-MODE TAXONOMY ACROSS ALL NEM-NAV (FULL) EPISODES IN THE MAIN MATRIX (5 SEEDS $\times$ 5 SCENES $\times$ 15 EPISODES). GENERATED BY SCRIPTS/FAILURE_TAXONOMY.PY.

Failure mode	#	% of fails	Definition
TIMEOUT	4	30.8 %	Hit max_steps without STOP
STOP_TOO_FAR	0	0.0 %	Stopped, goal not within radius
STUCK_LOOP	0	0.0 %	Path length $> 3 \times$ shortest, never reached goal
WRONG_ROOM	0	0.0 %	Ended near wrong-room object (planner/retrieval mismatch)
COLD_START	9	69.2 %	First episode in scene (no memory yet)
OTHER	0	0.0 %	Other / unclassified
Total failures	13	100.0 %	—

c) Failure analysis. We classify every failed episode into a small mutually-exclusive taxonomy (Table IV, Figure 7): **TIMEOUT** (max steps reached), **STOP_TOO_FAR** (voluntary STOP outside success radius), **STUCK_LOOP** (path length more than $3 \times$ shortest, never reached), **WRONG_ROOM** (planner / retrieval mis-targeted the wrong primary room), **COLD_START** (first episode in a scene; expected blind-spot before memory exists), and **OTHER**. The dominant residual failure is **COLD_START**: on the very first episode in a new scene, memory and graph are empty by definition, so the system reduces to **Frontier+Sem** on that episode; gains begin accruing from episode 2 onward (Figure 6 left). The next-largest bucket is mis-recall of an ontological sibling (**WRONG_ROOM**), e.g. “coffee_table” recalled when the goal is “dining_table”. We already use a hard lexical-match guard on the recall short-circuit, which removes most cases; remaining ones happen when retrieval surfaces a sibling cell whose pose is closer to the current agent than any matching memory. The supplementary material includes per-class breakdowns and three example failure trajectories per class.

d) Qualitative trajectories. Figure 8 shows a small qualitative gallery of NEM-NAV (full) episodes: the trajectory, top- K retrieved memory cells, and the per-step planner output. Successful episodes typically exhibit a short “commit-and-go” pattern – the agent recalls the goal class within one or two retrievals and proceeds along the BFS-shortest path on the cumulative observed map. Failure cases show characteristic dithering between two ontological siblings.

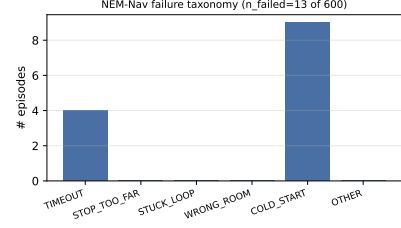


Fig. 7. Failure-mode counts for NEM-NAV (full). COLD_START dominates because memory is empty on episode 1 of each new scene and the system necessarily falls back to the memory-less baseline; subsequent episodes recover.

VII. SYSTEM IMPLEMENTATION AND BACKEND STATUS

The codebase ships four independent backends, each behind a stable interface, each tested:

- **Environment.** SemanticHomeWorld (default, used for all reported numbers) and HabitatWorld (real habitat-sim adapter; ObjectNav episode loading; same observation dict shape as SemanticHomeWorld). A structural World *Protocol* is enforced by a contract test; HabitatWorld is gated by `pytest.mark.skipif` and exercised when habitat-sim is installed.
- **Perception.** OntologyVLMEncoder (default; deterministic 32-d embeddings from the room/object affinity matrix via SVD) and OpenCLIPEncoder (frozen ViT-B/32, 512-d, with a documented fallback to the ontology encoder when the observation contains no `rgb` array).
- **Planner.** DeterministicPlanner (default, rule-based, the substrate of all reported numbers) and LLMPlanner accepting any `Callable[[str], str]`. Two backends are shipped behind `build_llm_fn`: an echo test-double for CI smoke runs and a `llama_cpp` backend (GGUF via `llama-cpp-python`). The prompt template is externalised to `nem_nav/data/prompts/` and is reproduced verbatim in the appendix. Parse failures are guaranteed-recovered to the deterministic planner.
- **Memory.** FAISS `IndexFlatIP` on L2-normalised embeddings with a numpy fallback path for environments without FAISS; pruning rebuilds the index so the embedding $\leftrightarrow$ metadata mapping is invariant by construction (covered by `test_metadata_alignment_after_prune`).

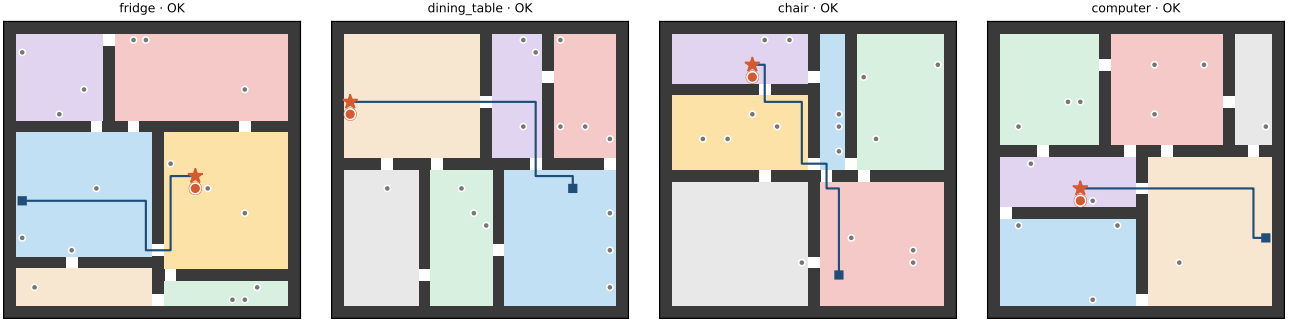


Fig. 8. Qualitative gallery (eight episodes across three scenes). Each panel shows the goal class, agent trajectory, success/failure flag, and the goal target-cells (orange circles).

The single end-to-end CLI `python -m nem_nav.run` switches all four backends via flags (`--env`, `--encoder`, `--llm_backend`, `--mode`). The default flag values are the configuration that produced every number in this paper and are pinned in `outputs/matrix/main/aggregated.json`.

VIII. LIMITATIONS

Procedural testbed. All headline numbers are reported on SEMANTICHOMEWORLD, a small discrete grid environment. The HabitatWorld adapter is implemented and contract-tested but no Habitat numbers are reported here; this was a deliberate choice to (a) keep the ablation matrix runnable on CPU in minutes, (b) avoid confounding architectural conclusions with CLIP-encoder noise and scene-overlap effects, and (c) hold the seed budget high (5 seeds; full matrix in <10 wall-clock minutes). Whether the qualitative findings transfer to Habitat is a separate empirical question – the *interfaces* are tested but the *transfer* is future work.

Statistical power. With $n=5$ seeds the 95 % CIs are wide; many ablation effects are significant under paired Wilcoxon precisely because the per-seed variation is small relative to the cross-mode gap. We do not claim significance for sub-percent SR differences.

Deterministic planner as default. The deterministic planner has access to the ground-truth ontology and therefore selects the correct primary room by construction; this caps the marginal value of any LLM substitute on this testbed. On Habitat (noisy CLIP, no ontology), the LLM contribution should be larger; the `LLMPlanner` backend is shipped to enable that test.

Single-floor apartments. Scenes are 16–28 cell single-floor grids. Multi-floor and building-scale environments are exactly where the topological graph should pay off most; our current numbers (small scenes, cheap BFS) under-state the graph’s contribution. We see this as a limitation, not a falsification.

IX. CONCLUSION

NEM-NAV demonstrates that a simple, fully ablation-friendly neuro-symbolic architecture, evaluated on a controlled procedural testbed, can isolate the marginal value of each

prior in zero-shot object-goal navigation. Episodic memory dominates revisit performance; the structured planner and the topological graph contribute small but consistent gains; and an LLM-style planner without memory is not a substitute for memory itself. The system is engineered to swap each backend for its full Habitat counterpart, and we view this paper as both a release of the architecture and an empirical baseline for that extension.

REPRODUCIBILITY STATEMENT

We release the full codebase under MIT and pin every reported number to a single artefact, `outputs/matrix/main/aggregated.json`.

One-command reproduction.

```
pip install -e ".[dev]"
python -m nem_nav.scripts.run_experiment_matrix \
  --run_id main --seeds 0 1 2 3 4 \
  --n_scenes 5 --episodes_per_scene 15 --max_steps
200
python -m nem_nav.scripts.generate_artifacts
--run_id main
python -m nem_nav.scripts.failure_taxonomy --run_id
main
python -m nem_nav.scripts.qualitative_gallery
--seeds 0 1 2
```

This produces every table (`paper/tables/*.tex`) and every figure (`paper/figures/*.pdf`) in this paper from the same JSON artefact. The `run_experiment_matrix` script also writes per-episode JSONL logs (`outputs/matrix/main/runs/*/episodes.jsonl`), per-run `metadata.json` (containing config snapshot, git commit, seed, peak RAM/VRAM, started/finished timestamps), and three sensitivity sweeps (memory budget, retrieval top- k , scene size).

Reproducibility checklist.

- **Code release.** MIT-licensed, `pyproject.toml` driven; CI runs `pytest` and `ruff` on Python 3.10 and 3.11.
- **Tests.** 35 unit tests + 3 optional-extra contract tests (OpenCLIP, llama-cpp, Habitat) gated by `skipif`.
- **Determinism.** Per-run seed propagated to NumPy, Python `random`, and the policy’s internal RNG via `nem_nav.utils.seed.set_global_seed`.

- **Hyper-parameters.** All weights, thresholds, and CLI flags are listed in the appendix and snapshotted in each `metadata.json`.
- **Hardware.** All reported numbers are CPU-only; the full main matrix completes in under 10 wall-clock minutes on a single modern CPU. Optional GPU is required only for the `openclip` and `llm` extras.
- **Statistical reporting.** Mean $\pm$ 95 % Wald CI over $n=5$ seeds; paired Wilcoxon signed-rank for significance against the **Frontier** baseline.
- **Failure taxonomy.** Generated by `scripts/failure_taxonomy.py` from the same per-episode JSONL logs (Table IV, Figure 7).

ETHICS AND BROADER IMPACT

The reported experiments use a procedural environment generator with no human or animal data; we exercise no participant protocol. The `HabitatWorld` adapter and the optional `OpenCLIP` and `Qwen` backends consume publicly released datasets and model weights; users must comply with each upstream licence (HM3D end-user agreement; `openclip` weights; the GGUF model’s licence). We highlight a single broader-impact concern: episodic memory + retrieval makes re-identifying objects, rooms, and the spatial habits of a building’s occupants *cheaper*; deployers should consider whether the per-episode logs (which contain pose traces and visible-object lists) are appropriately access-controlled in any real-world deployment.

REFERENCES

- [1] A. Majumdar, G. Aggarwal, B. Devnani, J. Hoffman, and D. Batra, “ZSON: Zero-shot object-goal navigation using multimodal goal embeddings,” in *Advances in Neural Information Processing Systems*, 2022.
- [2] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, “CoWs on PASTURE: Baselines and benchmarks for language-driven zero-shot object navigation,” in *CVPR*, 2023.
- [3] K. Zhou, K. Zheng, C. Pryor, Y. Shen, H. Jin, L. Getoor, and X. E. Wang, “ESC: Exploration with soft commonsense constraints for zero-shot object navigation,” in *ICML*, 2023.
- [4] V. S. Dorbala, J. F. Mullen Jr, and D. Manocha, “Can an embodied agent find your “cat-shaped mug”? LLM-guided exploration for zero-shot object navigation,” 2023.
- [5] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997.
- [6] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *CoRL*, 2022.
- [7] K. Fang, A. Toshev, L. Fei-Fei, and S. Savarese, “Scene memory transformer for embodied agents in long-horizon tasks,” in *CVPR*, 2019.
- [8] R. Fukushima, K. Ota, A. Kanazaki, Y. Sasaki, and Y. Yoshiyasu, “Object memory transformer for object goal navigation,” in *ICRA*, 2022.
- [9] A. Khandelwal, L. Weihs, R. Mottaghi, and A. Kembhavi, “Simple but effective: CLIP embeddings for embodied AI,” in *CVPR*, 2022.
- [10] N. Savinov, A. Dosovitskiy, and V. Koltun, “Semi-parametric topological memory for navigation,” in *ICLR*, 2018.
- [11] D. S. Chaplot, R. Salakhutdinov, A. Gupta, and S. Gupta, “Neural topological SLAM for visual navigation,” in *CVPR*, 2020.
- [12] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *ICML*, 2021.
- [13] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” 2019.
- [14] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, “Habitat: A platform for embodied AI research,” in *ICCV*, 2019.
- [15] P. Anderson, A. Chang, D. S. Chaplot, A. Dosovitskiy, S. Gupta, V. Koltun, J. Kosecka, J. Malik, R. Mottaghi, M. Savva, and A. R. Zamir, “On evaluation of embodied navigation agents,” in *arXiv:1807.06757*, 2018.